🔥 4. Jenkins Auto Build using GitHub (AI/DS Project)

🟦 Step 1: Start Jenkins (Docker)

```
docker start jenkins
docker ps
```

🟦 Step 2: Create AI Project

```
mkdir ai-ds-jenkins-project
cd ai-ds-jenkins-project
```

📄 **model.py**

```
print("AI/DS Jenkins Auto Build Running")
```

🟦 Step 3: Push to GitHub

```
git init
git remote add origin https://github.com/Mayuresh1705/ai-ds-jenkins-project.git
git add .
git commit -m "initial commit"
git push -u origin main
```

🟦 Step 4: Jenkins Job Configuration

In Jenkins UI:

- New Item → Freestyle Project → AI_DS_AutoBuild
- Source Code Management → Git
  Repo URL → your repo
  Branch → */main

🟦 Step 5: Build Step

Add:

```
echo "Build Started"
python3 model.py
echo "Build Finished"
```

🟦 Step 6: Trigger Build

```
echo print("update") >> model.py
git add .
```

```
git commit -m "trigger build"
git push origin main
```

✓ Build runs automatically (SCM/Webhook)

🔥 5. Docker Container for AI Model

□ Step 1: Create Project

```
mkdir ai-model-docker
cd ai-model-docker
```

📄 model.py

```
print("AI Model Running Successfully")

x = 10
y = 5
print("Prediction:", x + y)
```

📄 Dockerfile

```
FROM python:3.9

WORKDIR /app

COPY model.py .

CMD ["python", "model.py"]
```

□ Step 2: Build Image

```
docker build -t ai-model .
```

□ Step 3: Run Container

```
docker run ai-model
```

✓ Output:

```
AI Model Running Successfully
Prediction: 15
```

🔥 6. Docker Compose (AI + DB + Frontend)

□ Step 1: Create Structure

```
mkdir ai-docker-compose
cd ai-docker-compose
```

---

app.py (Backend)

```
from flask import Flask

app = Flask(__name__)

@app.route('/')
def home():
    return "AI Backend Running"

if __name__ == "__main__":
    app.run(host="0.0.0.0", port=5000)
```

Dockerfile

```
FROM python:3.9

WORKDIR /app

COPY app.py .

RUN pip install flask

CMD ["python", "app.py"]
```

---

docker-compose.yml

```
version: '3'

services:
  backend:
    build: .
    ports:
      - "5000:5000"

  frontend:
    image: nginx
    ports:
      - "3000:80"

  database:
```

```
  image: mysql:5.7
  environment:
    MYSQL_ROOT_PASSWORD: root
```

□ Step 2: Run

```
docker-compose up -d
```

□ Step 3: Stop

```
docker-compose down
```

🐳 7. Kubernetes (Minikube Deployment + Scaling)

□ Step 1: Start Minikube

```
minikube start
kubectl get nodes
```

□ Step 2: Use Minikube Docker

```
eval $(minikube docker-env)
docker build -t ai-model .
```

📄 model.py (Flask version)

```
from flask import Flask

app = Flask(__name__)

@app.route('/')
def home():
    return "AI Model Running on Kubernetes 🚀"

if __name__ == "__main__":
    app.run(host="0.0.0.0", port=5000)
```

📄 Dockerfile

```
FROM python:3.9

WORKDIR /app
```

```
COPY model.py .

RUN pip install flask

CMD ["python", "model.py"]
```

---

deployment.yaml

```yaml
apiVersion: apps/v1
kind: Deployment
metadata:
  name: ai-model-deployment

spec:
  replicas: 1
  selector:
    matchLabels:
      app: ai-model

  template:
    metadata:
      labels:
        app: ai-model

    spec:
      containers:
      - name: ai-model
        image: ai-model
        imagePullPolicy: Never
        ports:
        - containerPort: 5000
```

Step 3: Deploy

```
kubectl apply -f deployment.yaml
kubectl get pods
```

Step 4: Expose Service

```
kubectl expose deployment ai-model-deployment --type=NodePort --port=5000 --target-port=5000
```

Step 5: Access

```
minikube service ai-model-deployment --url
```

□ Step 6: Scale ☆

```
kubectl scale deployment ai-model-deployment --replicas=3
kubectl get pods
```

🔥 8. Kubernetes Auto-Scaling + Monitoring

□ Step 1: Enable Metrics Server

```
minikube addons enable metrics-server
```

□ Step 2: Create Autoscaler

```
kubectl autoscale deployment ai-model-deployment --cpu-percent=50 --min=1 --max=5
```

□ Step 3: Check HPA

```
kubectl get hpa
```

□ Step 4: Monitor Usage

```
kubectl top pods
kubectl top nodes
```

□ Step 5: Generate Load

```
while true; do curl $(minikube service ai-model-deployment --url); done
```

🛑 FINAL STOP (ALL)

```
kubectl delete deployment ai-model-deployment
kubectl delete service ai-model-deployment
minikube stop

docker-compose down
docker stop $(docker ps -q)
docker rm $(docker ps -aq)
docker system prune -f
```